Lecture Notes: Variational Auto Encoders and Generative Adversarial Networks

Course: 2110573 Pattern Recognition

2026-03-19
version 1.0

1 Introduction to Generative Modeling

Generative modeling is a branch of machine learning that focuses on learning the underlying distribution of a dataset in order to generate new, synthetic data points that are similar to the real data.

- **Core Objective:** The goal is to learn a joint probability distribution $P(X, Y)$ or a conditional probability distribution $P(X|Y)$.
- **Sampling:** Once the model has learned the distribution, it can sample (generate) new instances X from $P(X|Y)$, where Y acts as a controlling parameter (e.g., a class label or a latent vector).
- **Distribution Modeling:** $P(X|Y)$ is often assumed to be a parametric distribution.

Generative models can be broadly classified based on how they handle the probability density function $P(X)$ [5]:

1. **Explicit Density Models:** These models explicitly define and maximize the likelihood $P(X)$.
   - *Tractable Density:* Fully visible belief networks (NADE [19] , MADE [4] , PixelRNN [20]), Change of variables models (Nonlinear ICA). Autoregressive language models (LLMs) [16] also fall here.
   - *Approximate Density:* in cases where the probabilistic model is too complicated. A simpler distribution is used to approximate the actual distribution. There three main camps aims camps to approximate such distributions. 1) Variational methods such as Autoencoders 2) Markov Chain approximations such as Boltzmann machines 3) Diffusion modeling such as SDE [17], DDPM [10], SGM [18].
2. **Implicit Density Models:** These models learn to generate data without explicitly defining $P(X)$.
   - *Direct approaches:* Generative Adversarial Networks (GANs).
   - *Markov Chain approaches:* Generative Stochastic Networks (GSN).

This lecture focuses primarily on deep generative visual models, specifically **Autoencoders, Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs)**.

2 Autoencoders (AE)

Deep Neural Networks (DNNs) are powerful representation learning models capable of learning highly complex, non-linear mappings. An autoencoder [2] leverages this capability to compress and reconstruct data.

2.1 Standard Autoencoder Architecture

An autoencoder learns to copy its input to its output ("auto" means self). It consists of two main parts:

1. **Encoder:** Compresses the input X into a lower-dimensional latent representation (or code) Z.
2. **Decoder:** Reconstructs the input from the latent code Z to produce the output X'.

The network is typically trained using an L2 (Mean Squared Error) loss function to minimize the difference between the input and the reconstructed output:

$$\mathcal{L} = ||X - X'||^2 \tag{1}$$

By inspecting the latent attributes learned by the autoencoder, one can perform interpolations. Varying specific dimensions of the latent code Z can result in smooth transitions of semantic attributes in the decoded image (e.g., adding sunglasses, changing hair color).

2.2 Limitations of Standard Autoencoders

Standard AEs learn a **deterministic mapping** from $X \to Z \to X'$. This point-estimate approach poses a problem for generation. If we sample a random point from the latent space Z that the model has never seen, the deterministic decoder may map it to an improbable, nonsensical image.

To generate new, realistic samples, we need a **non-deterministic mapping**, which requires learning the underlying probability distribution of the latent space.

3 Variational Autoencoders (VAE)

3.1 Motivations

Variational Autoencoders [14] introduce probabilistic elements to autoencoders. The motivations are threefold:

1. **Learning a non-deterministic mapping:** Instead of outputting a single vector, the encoder outputs the parameters of a probability distribution.
2. **Solving Intractable Inference:** It provides a method to approximate intractable posterior distributions.
3. **Generating new data:** By forcing the latent space to follow a known prior distribution (like a standard Gaussian), we can easily sample from it to generate new images using the decoder.

3.2 Probabilistic Autoencoder Formulation

In a VAE, the encoder does not output a fixed vector Z. Instead, it outputs the parameters (mean μ and standard deviation σ) of a Gaussian distribution. We then sample Z from this distribution:

$$Z \sim \mathcal{N}(\mu, \sigma^2) \tag{2}$$

This latent code Z is then passed to the decoder to generate the image.

3.3 The Bayesian Perspective and Intractable Inference

In Bayesian terms, we observe data $\mathbf{x}$ and want to infer the latent variables $\mathbf{z}$. We are interested in the true posterior distribution $p(\mathbf{z}|\mathbf{x})$. By Bayes' rule:

$$p(\mathbf{z}|\mathbf{x}) = \frac{p(\mathbf{z}, \mathbf{x})}{p(\mathbf{x})} = \frac{p(\mathbf{z}, \mathbf{x})}{\int p(\mathbf{x}, \mathbf{z}) d\mathbf{z}} \tag{3}$$

The denominator (model evidence or marginal likelihood) requires an integral over all possible configurations of $\mathbf{z}$, which is computationally intractable in high-dimensional spaces.

3.4 Variational Inference and ELBO

To solve this, VAEs use Variational Inference. We approximate the true posterior $p(\mathbf{z}|\mathbf{x})$ with a simpler, tractable distribution $q(\mathbf{z}|\mathbf{x})$ (modeled by the encoder network).

We find the best approximation by minimizing the Kullback-Leibler (KL) divergence between our approximation and the true posterior:

$$q^*(\mathbf{z}|\mathbf{x}) = \arg\min_q \mathrm{KL}[q(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] \tag{4}$$

By expanding the KL divergence definition, we get:

$$\begin{aligned} \mathrm{KL}[q(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] &= \mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{z}|\mathbf{x})] & (5)\\ &= \mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{z},\mathbf{x})] + \log p(\mathbf{x}) & (6) \end{aligned}$$

Rearranging for $\log p(\mathbf{x})$:

$$\log p(\mathbf{x}) = \mathrm{KL}[q(\mathbf{z}|\mathbf{x})||p(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z},\mathbf{x})] \tag{7}$$

Since KL divergence is always non-negative ($\mathrm{KL} \geq 0$), we can establish a lower bound on the log-likelihood of the data:

$$\log p(\mathbf{x}) \geq -\mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z},\mathbf{x})] \equiv \text{ELBO (Evidence Lower Bound)} \tag{8}$$

Minimizing the KL divergence to the intractable posterior is mathematically equivalent to **maximizing the ELBO**.

3.5 The VAE Loss Function

Let's break down the ELBO objective to construct our loss function:

$$\begin{aligned} \max_q \mathrm{ELBO} &= \min_q \mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z},\mathbf{x})] & (9)\\ &= \min_q \mathbb{E}_q[\log q(\mathbf{z}|\mathbf{x})] - \mathbb{E}_q[\log p(\mathbf{x}|\mathbf{z})] - \mathbb{E}_q[\log p(\mathbf{z})] & (10)\\ &= \min_q \underbrace{\mathrm{KL}[q(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]}_{\text{Regularization Term}} - \underbrace{\mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Term}} & (11) \end{aligned}$$

1. **Data Log Likelihood (Reconstruction Term):** This term encourages the decoder to accurately reconstruct x from z. If we model $p(\mathbf{x}|\mathbf{z})$ as a Gaussian where the mean is the output of the decoder, maximizing this log-likelihood is mathematically equivalent to minimizing the standard L2 (Mean Squared Error) loss between X and X'.
2. **KL Divergence (Regularization Term):** This term forces the approximate posterior $q(\mathbf{z}|\mathbf{x})$ (the encoder's output) to be close to a prior distribution $p(\mathbf{z})$, usually chosen to be a standard unit Gaussian $\mathcal{N}(0, I)$. This ensures the latent space is continuous, clustered around the origin, and allows for smooth interpolation and sampling.

The final VAE Loss is roughly:

$$\mathcal{L}_{\mathrm{VAE}} \approx ||X - X'||^2 + \mathrm{KL}[q(\mathbf{z}|\mathbf{x})||\mathcal{N}(0, I)] \tag{12}$$

3.6 Computing the KL Divergence

To optimize the VAE loss in practice, we need an analytical solution for the KL divergence term so that we don't have to estimate it via sampling. We typically assume that the prior $p(\mathbf{z})$ is a standard normal distribution $\mathcal{N}(0, \mathbf{I})$ and the approximate posterior $q(\mathbf{z}|\mathbf{x})$ is a multivariate Gaussian with a diagonal covariance matrix $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ produced by the encoder.

Under these assumptions, the KL divergence between the two Gaussians can be computed analytically using the following closed-form equation:

$$\mathrm{KL}[\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)||\mathcal{N}(0, \mathbf{I})] = -\frac{1}{2}\sum_{j=1}^{J}\left(1 + \log((\sigma_j)^2) - (\mu_j)^2 - (\sigma_j)^2\right) \tag{13}$$

where J is the dimensionality of the latent space, and μ_j and σ_j are the mean and standard deviation of the j-th dimension of the latent vector. Because this equation is fully deterministic with respect to the encoder's outputs (μ and σ), it is highly efficient to compute and easily differentiable for backpropagation.

3.7 The Reparameterization Trick

To train the VAE using Stochastic Gradient Descent (SGD), we must compute gradients with respect to the network parameters. However, the latent code $\mathbf{z}$ is drawn from a random sampling process ($z \sim \mathcal{N}(\mu, \sigma)$), and **backpropagation cannot flow through a random node**.

To solve this, we use the **Reparameterization Trick**. Instead of sampling z directly, we sample noise ϵ from a standard Gaussian and scale/shift it deterministically using the network outputs:

$$\epsilon \sim \mathcal{N}(0, I) \quad \Longrightarrow \quad \mathbf{z} = \mu + \sigma \odot \epsilon \tag{14}$$

Now, the randomness is isolated in ϵ, and gradients can flow deterministically through μ and σ back to the encoder.

4 Advanced Autoencoder Concepts

4.1 β-VAE (Disentanglement)

Researchers often want a compact, meaningful representation where individual dimensions of the latent code Z map to distinct, interpretable human concepts (e.g., z_1 = hair color, z_2 = glasses, z_3 = age). This is called **disentangled representation**. β-VAE [9, 3] achieves this by adding a hyperparameter β to heavily penalize the KL divergence term, forcing the network to be more efficient and independent in how it utilizes the latent dimensions.

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})] - \beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{15}$$

4.2 VQ-VAE (Vector Quantized VAE)

Instead of learning continuous latent variables, VQ-VAE learns **discrete** latent representations using vector quantization.

- The encoder produces a grid of vectors (e.g., 32×32 vectors).
- These vectors are mapped to the nearest vector (codeword) inside a learned **Codebook**.
- Because the representation is discrete, it avoids the problem of "posterior collapse" seen in continuous VAEs and is highly effective when paired with autoregressive models (like Transformers in DALL-E) to model the prior over the discrete codes.

VQ-VAE Overall Loss Formulation [21]:

$$L = \underbrace{\log p(x|z_q(x))}_{\text{Reconstruction}} + \underbrace{||\text{sg}[z_e(x)] - e||_2^2}_{\text{Codebook Loss}} + \underbrace{\beta||z_e(x) - \text{sg}[e]||_2^2}_{\text{Commitment Loss}} \tag{16}$$

Note: sg[...] stands for "stop gradient" (e.g., `.detach()` in PyTorch), which prevents gradient flow through that specific term during backpropagation.

4.3 Perceptual Loss

Standard pixel-wise L2 reconstruction loss often produces blurry images because it fails to capture high-level semantic content. **Perceptual Loss** [11] measures semantic differences instead. Images are passed through a pre-trained image classifier (like VGG). The loss is calculated as the L2 distance between the intermediate feature maps extracted from the classifier, ensuring the generated image has similar semantic features to the ground truth.

$$\ell_{feat}^{\phi,j}(\hat{y}, y) = \frac{1}{C_j H_j W_j} ||\phi_j(\hat{y}) - \phi_j(y)||_2^2 \tag{17}$$

5 Generative Adversarial Networks (GAN)

If a VAE is like a single network learning to map and decode distributions, a GAN [7, 6] operates on the principle of competition.

5.1 The Minimax Game

Consider a money counterfeiter and a police officer:

- **Generator (G):** The counterfeiter taking random noise $\mathbf{z} \sim \mathcal{N}(0, I)$ and generating fake images $\mathbf{x}_{fake}$ to fool the police.
- **Discriminator (D):** The police trying to classify whether an input image $\mathbf{x}$ is real (1) or fake (0).

The training process is a **minimax game** where G tries to maximize the probability of D making a mistake, while D tries to maximize its accuracy.

Discriminator Objective (Maximize):

$$\max_D \mathbb{E}_{\mathbf{x}\sim p_{data}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))] \tag{18}$$

This is equivalent to standard binary cross-entropy loss.

Generator Objective (Minimize):

$$\min_G \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))] \tag{19}$$

Combining them yields the overall Minimax formulation:

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x}}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}}[\log(1 - D(G(\mathbf{z})))] \tag{20}$$

5.2 Conditional GANs (cGAN)

To generate images of a specific class or with a certain trait, we can provide a condition vector $\mathbf{c}$ (e.g., class label) to both the Generator and the Discriminator. G takes $[\mathbf{z}, \mathbf{c}]$ as input, and D takes $[\mathbf{x}, \mathbf{c}]$ as input. D must not only verify if the image is real but also if it matches the requested class $\mathbf{c}$.

5.3 Challenges in GANs

Mode Collapse: The Generator may discover that producing a single, highly realistic type of image (a single "mode") is enough to consistently fool the Discriminator. Consequently, the model fails to learn the diversity of the entire data distribution and outputs the same limited set of images regardless of the random input.

5.4 Advancements in GAN Architectures

- **Progressive GANs [12]:** Training starts at a very low resolution (e.g., 4×4) and progressively adds new layers to both G and D to increase the resolution up to 1024×1024. This stabilizes training significantly.
- **StyleGAN [13]:** Built upon Progressive GANs, it borrows concepts from style transfer. Instead of feeding the latent vector $\mathbf{z}$ directly into the input layer, $\mathbf{z}$ is passed through a mapping network to generate "styles," which then modulate the generator at various resolutions using Adaptive Instance Normalization (AdaIN).

6 Wasserstein GAN (WGAN)

Vanilla GAN minimizes the Jensen-Shannon (JS) divergence between the real and generated distributions, which can lead to vanishing gradients if the distributions do not overlap.

WGAN [1] introduces the **Wasserstein Distance** (or Earth Mover's Distance), which measures the minimal energy required to transport mass to transform one distribution into another. This distance provides a smooth gradient even when distributions are disjoint.

- In WGAN, the Discriminator is replaced by a **Critic**. It outputs a raw score (linear activation) instead of a probability via a sigmoid function.
- To approximate the Earth Mover's Distance mathematically, the Critic function must be **1-Lipschitz continuous** (its gradient norm must be bounded by 1 everywhere).

- **WGAN Weight Clipping:** The original WGAN enforced the Lipschitz constraint by capping network weights to a small range (e.g., $[-0.01, 0.01]$).
- **WGAN-GP (Gradient Penalty) [8]:** Weight clipping leads to poor capacity. WGAN-GP improves this by adding a penalty to the loss function that directly penalizes the model if the gradient norm of the Critic moves away from 1.

WGAN-GP Loss:

$$L = \underbrace{\mathbb{E}_{\tilde{x}\sim P_g}[D(\tilde{x})] - \mathbb{E}_{x\sim P_r}[D(x)]}_{\text{Original Critic Loss}} + \underbrace{\lambda \mathbb{E}_{\hat{x}\sim P_{\hat{x}}}[(||\nabla_{\hat{x}} D(\hat{x})||_2 - 1)^2]}_{\text{Gradient Penalty}} \tag{21}$$

7 GAN Applications

GANs are highly versatile beyond basic image generation:

- **StackGAN [23]:** Generating photo-realistic images from text descriptions.
- **SRGAN (Super-Resolution GAN) [15]:** Enhancing low-resolution images to high-resolution while hallucinating realistic fine details.
- **Content-Aware Filling (Inpainting):** Intelligently filling missing or masked patches in an image by understanding global context.
- **CycleGAN (Style Transfer) [24]:** Translating images from one domain to another (e.g., Horses $\leftrightarrow$ Zebras) *without paired data*. It utilizes a **Cycle Consistency Loss** (translating A to B, and back to A must reconstruct the original image) to ensure the structure is maintained during style transfer.
- **Adversarial Debiasing [22]:** Using adversarial concepts to remove bias from latent representations. An encoder learns representations, while an adversary (a classifier) tries to predict a protected attribute (like clothes color). The encoder is trained to maximize the adversary's cross-entropy loss, forcing it to drop the biased information.

8 Summary: GAN vs. VAE

The choice between GANs, VAEs, and modern Diffusion models often depends on the **Generative Learning Trilemma**: balancing High Quality, Fast Sampling, and Mode Coverage/Diversity.

- **GANs:**
  - **Pros:** Produce sharper, higher-quality images. Fast sampling.
  - **Cons:** Notoriously difficult to train (unstable minimax game), prone to mode collapse (poor diversity), does not possess a natural encoder for inference.
- **VAEs:**
  - **Pros:** Highly stable training, excellent mode coverage (diversity), learns a disentangled, meaningful latent space allowing for smooth interpolation, and provides an encoder for downstream tasks.
  - **Cons:** Outputs tend to be blurrier than GANs due to the L2 reconstruction loss and prior regularization.
- **Modern Context:** Today, many state-of-the-art models are based on **Diffusion Models**, which synthesize tricks and structural architectures derived from years of GAN and VAE research.

References

[1] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein generative adversarial networks. In *International conference on machine learning*, pages 214–223. Pmlr, 2017.

[2] Pierre Baldi. Autoencoders, unsupervised learning, and deep architectures. In *Proceedings of ICML workshop on unsupervised and transfer learning*, pages 37–49. JMLR Workshop and Conference Proceedings, 2012.

[3] Christopher P. Burgess, Irina Higgins, Arka Pal, Loic Matthey, Nick Watters, Guillaume Desjardins, and Alexander Lerchner. Understanding disentangling in β-vae, 2018.

[4] Mathieu Germain, Karol Gregor, Iain Murray, and Hugo Larochelle. Made: Masked autoencoder for distribution estimation. In *International conference on machine learning*, pages 881–889. PMLR, 2015.

[5] Ian Goodfellow. Nips 2016 tutorial: Generative adversarial networks, 2017.

[6] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020.

[7] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

[8] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C Courville. Improved training of wasserstein gans. *Advances in neural information processing systems*, 30, 2017.

[9] I Higgins, L Matthey, A Pal, C Burgess, X Glorot, M Botvinick, S Mohamed, and A Lerchner. Beta-vae: Learning basic visual concepts with a constrained variational framework (iclr), 2017.

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[11] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European conference on computer vision*, pages 694–711. Springer, 2016.

[12] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation. *arXiv preprint arXiv:1710.10196*, 2017.

[13] Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4401–4410, 2019.

[14] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.

[15] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al. Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017.

[16] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey, 2025.

[17] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

[18] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[19] Benigno Uria, Iain Murray, and Hugo Larochelle. A deep and tractable density estimator. In *International Conference on Machine Learning*, pages 467–475. PMLR, 2014.

[20] Aäron Van Den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International conference on machine learning*, pages 1747–1756. PMLR, 2016.

[21] Aaron Van Den Oord, Oriol Vinyals, et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.

[22] Brian Hu Zhang, Blake Lemoine, and Margaret Mitchell. Mitigating unwanted biases with adversarial learning. In *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*, pages 335–340, 2018.

[23] Han Zhang, Tao Xu, Hongsheng Li, Shaoting Zhang, Xiaogang Wang, Xiaolei Huang, and Dimitris N Metaxas. Stackgan: Text to photo-realistic image synthesis with stacked generative adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 5907–5915, 2017.

[24] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2223–2232, 2017.